

category: camera-integration

tags: [channel, async, dispose, freeze, winforms, ffmpeg, reconnect]

severity: high

created: 2026-07-22

updated: 2026-07-22

project-origin: Kztek.Camera BaseLIB (WinForms, 1.Source/Kztek.Cameras)

Channel<T> consumer bị stuck mãi mãi nếu `Dispose()` không gọi `TryComplete()` — camera WinForms đứng hình khi RTSP reconnect

Tình huống gặp phải

Điều tra bug "camera WinForms thỉnh thoảng bị đứng hình" ([AnvPlayer](#) UserControl). Camera ngừng hiển thị hình sau khi RTSP mất kết nối và tự kết nối lại. Luồng RTSP thực sự đã phục hồi (decode đang chạy, frames vào channel mới) nhưng UI không cập nhật cho đến khi user bấm Stop → Start thủ công.

Triệu chứng / Lỗi

- Control hiển thị "Connecting..." hoặc frame tĩnh cuối cùng vô thời hạn.
- Không có exception nào hiển thị (catch block rỗng).
- Chỉ hết khi gọi `Stop()` + `Start()` lại (reset `CancellationToken`).
- Xảy ra ngẫu nhiên, tần suất phụ thuộc tần suất RTSP drop (camera reboot, network hiccup).

Nguyên nhân gốc rễ (Root Cause)

`VideoStreamDecoderIntPtr.Dispose()` không gọi `TryComplete()` trên các `Channel<T>` writers.

Khi RTSP drop, `PollingDecodeFrameAsync` bắt exception và reconnect theo vòng lặp:

```
while (!token.IsCancellationRequested)
{
    try
    {
        _decoder?.Dispose(); // ← Dispose old decoder, nhưng KHÔNG complete channels
        _decoder = new VideoStreamDecoderIntPtr(...); // ← New decoder, new channels
        // ...
    }
    catch { await Task.Delay(1000, token); }
```

Trong khi đó, `ConsumerDisplayFrameAsync` đang await:

```
await foreach (var packet in decoder.FrameChannel.Reader.ReadAllAsync(token))
```

`ReadAllAsync` chờ đến khi có item HOẶC channel writer completed HOẶC token cancelled.

Vì `TryComplete()` không được gọi → channel writer của decoder Cũ vẫn "mở" → `ReadAllAsync` bị stuck mãi mãi. Token chưa bị cancel (chỉ cancel khi `Stop()` được gọi thủ công).

New decoder đã push frames vào NEW channel, nhưng consumer vẫn đang chờ ở OLD channel.

→ Màn hình đứng hình.

Tương tự với `ConsumerMotionDetectionAsync` (MdChannel) và `ConsumerAiDetectionAsync` (AiChannel).

Giải pháp

Thêm 3 dòng `TryComplete()` vào ĐẦU `Dispose()`, TRƯỚC khi dọn bitmaps/FFmpeg:

```
public void Dispose()
{
    // 0) Complete channels TRƯỚC — consumer thoát ReadAllAsync ngay, không cần chờ
    token cancel
    FrameChannel.Writer.TryComplete();
    MdChannel.Writer.TryComplete();
    AiChannel.Writer.TryComplete();

    // 1) Dọn queue bitmap, FFmpeg resources...
    // ...
}
```

Sau khi `TryComplete()`:

- `ReadAllAsync` drain hết items còn lại trong channel rồi trả về (không block).
- Consumer loop thoát, `var decoder = _service.Decoder` lấy decoder mới, subscribe channel mới.
- UI cập nhật bình thường.

Áp dụng lại (How to reuse)

- Bất kỳ lúc nào dùng `Channel<T>` với pattern `await foreach (ReadAllAsync(token))` trong vòng lặp có thể dispose + tạo lại channel source: **PHẢI** gọi `Writer.TryComplete()` trong `Dispose()`.
- Dấu hiệu nhận biết bug này: UI "stuck" sau reconnect, không exception, chỉ hết khi reset CTS → nghi ngờ ngay channel consumer chưa được signal `Complete`.
- Debug nhanh: đặt breakpoint/log ở đầu vòng lặp `ConsumerXxxAsync` — nếu không bao giờ loop lại sau reconnect → channel cũ không bao giờ complete.

Chú ý / Cạm bẫy (Gotchas)

- ⚠ `TryComplete()` phải được gọi TRƯỚC khi dọn bitmaps/native resources để tránh race condition: consumer có thể đang xử lý item cuối, nếu native resources bị free trước, consumer dùng bitmap đã free → crash. Thứ tự: complete → drain (tự động) → rồi mới cleanup.
- ⚠ `TryComplete()` là idempotent — gọi nhiều lần không có hại. An toàn gọi ngay cả khi channel đã complete hoặc chưa có writer nào (channel có thể còn item chưa đọc — consumer sẽ drain hết trước khi vòng lặp `ReadAllAsync` kết thúc).

- ⚠ Khác với `Writer.Complete(Exception)` (complete với lỗi): `TryComplete()` không throw, không propagate exception vào consumer. Consumer đọc hết items rồi thoát bình thường.
- ⚠ Nếu consumer đang await `ReadAsync()` (không phải `ReadAllAsync`), cần kiểm tra `ChannelClosedException` thay vì tự động thoát — `ReadAllAsync` xử lý điều này trong vòng lặp.
- ⚠ Pattern này đặc biệt dễ bị bỏ sót khi channel được khởi tạo trong constructor (không phải trong `Start()/Connect()`) → developer hay quên không "pair" `TryComplete` trong `Dispose`.

Tham chiếu

- `Kztek.Camera/1.Source/Kztek.Cameras/Players/FFMPEG/UserControls/VideoStreamDecoderIntptr.cs`
— `Dispose()`, `FrameChannel`, `MdChannel`, `AiChannel`
- `Kztek.Camera/1.Source/Kztek.Cameras/Players/FFMPEG/UserControls/AnvPlayer.cs`
— `ConsumerDisplayFrameAsync()` (awaiting `FrameChannel.Reader.ReadAllAsync`)
- `Kztek.Camera/1.Source/Kztek.Cameras/Players/FFMPEG/UserControls/AnvPlayerService.cs`
— `ConsumerMotionDetectionAsync()`, `ConsumerAiDetectionAsync()`, `PollingDecodeFrameAsync()`
- Bug report: [docs/bugs/BUG-camera-winforms-freeze.md](#) (BUG-WF-001)